全局变量为什么自动是 0：.data 与 .bss 的分工

嵌入式 Linux 手记　2026-11-05　公众号文章 · 原链接 mp.weixin.qq.com

先猜输出。这段程序在 Linux 上打印什么？

```c
#include <stdio.h>

int g;              /* 没写初始化 */
int g2 = 42;

int main(void) {
    int l;          /* 也没写初始化 */
    printf("g = %d\n", g);
    printf("g2 = %d\n", g2);
    printf("l = %d\n", l);   /* 这行危险 */
    return 0;
}
```

`g` 打印 0，`g2` 打印 42，`l` 打印一个不可预测的垃圾值。同样是"没写初始化"，全局变量 `g` 与局部变量 `l` 的待遇天差地别。这不是编译器的心情，是 C 标准的明文规定，落地则靠两个段的名字：`.data` 与 `.bss`。这一篇把"全局变量为什么自动是 0"从标准条文讲到 ELF 文件布局，再到裸机启动代码里那两段你抄过无数遍的循环。

一、标准怎么规定的：两种存储期，两套规矩

C11 §6.7.9 第 10 条：静态存储期的对象（全局变量、`static` 变量）如果没有显式初始化，执行零初始化：算术类型为 0，指针为 NULL。这是语言的硬承诺，每次程序启动都兑现。

自动变量（局部）走的是另一条：不初始化则值不确定（indeterminate），读它是未定义行为（读未初始化的专题在篇 41，UB 家族另见篇 8）。所以 `l` 打印出来的不是"随机数"这么温柔，标准连"它是个数"都不保证。

待遇差异的合理性藏在成本里：全局变量一生只有一份、程序启动只初始化一次，清零成本可以忽略；局部变量每次函数调用都开新帧，逐帧清零是给所有函数加一笔常驻税，而大多数局部变量在下一次赋值前根本不需要初值——标准把这笔税免了，代价是"值不确定"的责任转到程序员头上。



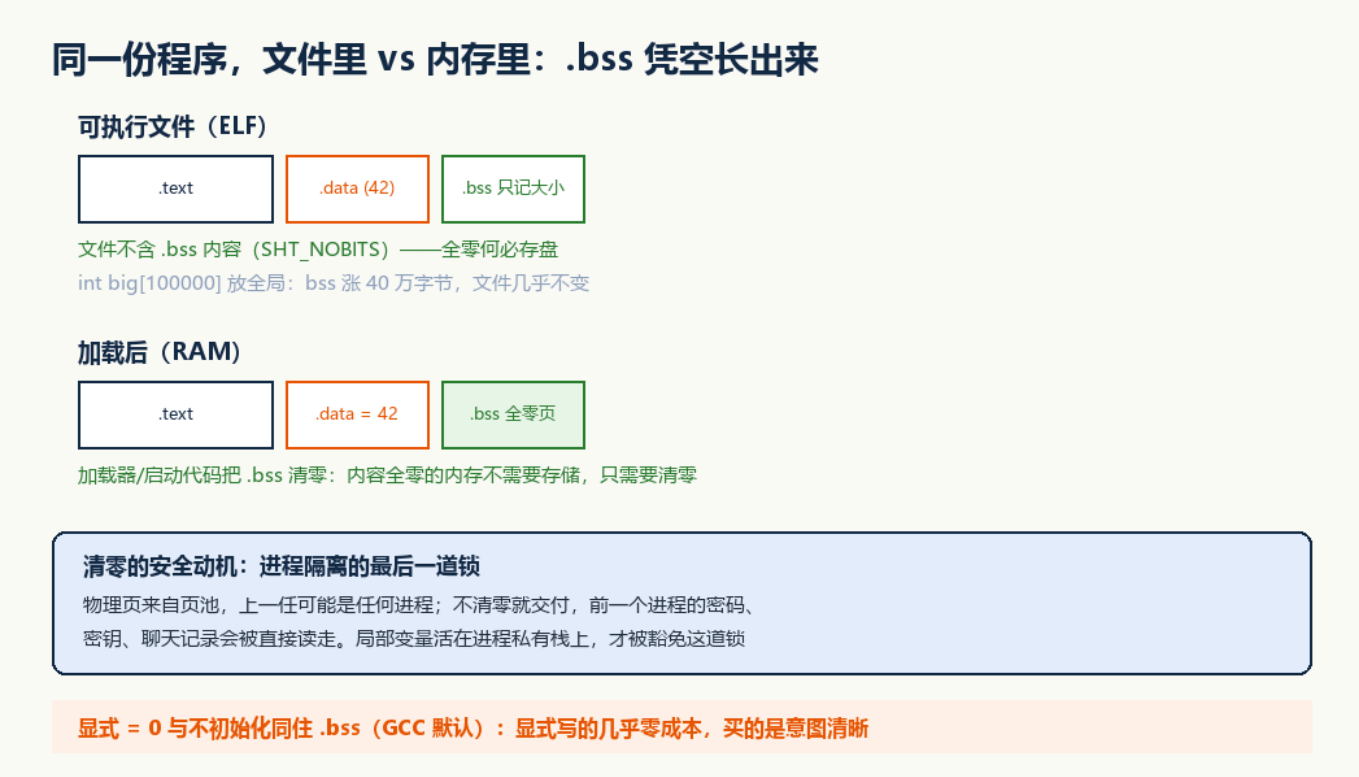
图：三个变量三种归宿。零初始化、原样进文件、值不确定——标准条文与段级分工一一对齐。

二、ELF 视角：.data 与 .bss 的分工

到了可执行文件层面，"全局"拆成了两个段，分工精确到字节。

`.data` 段存放已初始化的全局/静态变量：`g2 = 42` 的 42 必须原样写进文件，加载器不知道你想初始化成几，只有文件自己知道。.bss 段存放未初始化（含显式 = 0）的全局/静态变量：它在文件里只登记一个名字和大小（ELF 段类型 `SHT_NOBITS`，"无比特"），内容一个字节都不占。反正全是零，何必在磁盘上存一堆 0？加载时由系统把这段内存清零即可。

两个直接可观察的证据：其一，用 `size a.out` 看编译产物，`bss` 那一列的数字不增加文件大小：`int big[100000];` 放全局只让 bss 涨 40 万字节，文件几乎不变；其二，篇 16 的 `malloc` 大块为什么按需分配零页，与本篇 `.bss` 为什么不占文件，是同一个经济学：内容全零的内存，不需要存储，只需要清零。



图：同一份程序，文件里与内存里的对照。.bss 在文件中只记大小，加载后凭空"长"成全零页；清零是进程隔离的最后一道锁。

裸机玩家对这套分工体感最深。裸机的账更直观：没有操作系统与加载器，`.data` 的初值由链接脚本安排在 Flash 里的固定位置，启动代码按段起址逐字搬运——镜像里存的是"烧进 Flash 的初值"，RAM 里才是"变量真正住的地方"，两个世界的尺寸差就是 `.bss` 的省出来的体积。

Cortex-M 的启动代码（startup 文件）里固定有两段循环：把 `.data` 的初值从 Flash 拷到 RAM，把 `.bss` 逐字节清零，C 运行环境的一半就是这两个循环搭起来的。篇 21 讲 `xTaskCreateStatic` 时说"链接期锁死全部内存"，MAP 文件里 `.data` 与 `.bss` 各占多少、谁在 Flash 谁只在 RAM，账目同样出自这套分工。

三、清零的安全动机

`.bss` 清零不只是给 C 标准的零初始化兜底，还是一道安全边界。操作系统给新进程分配的物理页来自页池，上一任使用者可能是任何进程，如果不清零就交付，前一个进程留下的密码、密钥、聊天记录会被下一个进程直接读走。篇 16 讲过匿名映射的物理页补齐时填零，动机正在这里：零初始化是进程隔离的最后一道锁，宁可花清零的时间，也不让数据跨进程串门。

局部变量没有这道锁，因为它活在栈上，而栈页是进程自己的，脏值虽然难看，泄露不出进程边界（被优化器放大成 UB 是另一回事，篇 8 已立案）。

四、两个工程口径

口径一，显式 `= 0` 与不初始化在主流工具链里同住 `.bss`：GCC 默认把显式写 0 的全局变量也归入 bss（想强制进 `.data` 用 `-fno-zero-initialized-in-bss`）。所以显式写 `= 0` 几乎零成本，收益是意图清晰："我要的就是 0"写在代码里，读代码的人不用回忆标准第几条。

口径二，`g` 自动为 0 不等于"可以放心用所有没初始化的东西"：指针全局确实自动为 NULL（零初始化含指针），判空逻辑仍要写；局部数组想清零，`= {0}` 或 `memset` 自己来；跨过这些边界去读未初始化的自动变量，编译器的优化（篇 8 的 UB 折叠）会让"看起来能跑"变成"随时不能跑"。

写在最后

1. 静态存储期对象未显式初始化执行零初始化（C11 §6.7.9），全局与 `static` 变量开机即 0 是硬承诺；自动变量值不确定，读它是 UB。
2. `.data` 存初值占文件空间，`.bss` 只记大小不占文件（SHT_NOBITS）：内容全零的内存不需要存储，只需要清零，与篇 16 的零页同一经济学。
3. 裸机启动代码的两个循环（拷 `.data`、清 `.bss`）就是在人工执行这套分工；`size` 命令可看两段账目。
4. 清零是安全边界：物理页交付前填零，防上一进程的数据跨进程泄漏；局部变量活在进程私有的栈上，才被豁免。
5. 显式 `= 0` 几乎零成本（同住 `.bss`），买的是意图清晰；指针全局自动为 NULL，但判空逻辑照写。

段的分工讲完了，下一站先解决"内存怎么摆"的问题：为什么 `struct{char;int}` 会占 8 个字节。下一篇《struct{char;int} 为什么占了 8 字节：对齐与 padding》，拆开对齐与填充的账本。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。